PATHCAST: A Formally Specified Pipeline for Process-Aware Stochastic Forecasting of the Software Development Lifecycle

Rodrigo Almeida de Oliveira[a,1,*], Juliano de Paulo Ribeiro[a,2], Edson Emilio Scalabrin[a,3]

[a]*Pontifícia Universidade Católica do Paraná (PUCPR), Graduate Program in Informatics (PPGIa), Curitiba, PR, Brazil*

Abstract

Context. Software repositories continuously produce event data — commits, issue transitions, pull requests, CI/CD runs — that process mining (PM) and stochastic modeling can in principle exploit for software development lifecycle (SDLC) forecasting. A recent systematic literature review, however, found that only one of 404 confirmed primary studies jointly integrates process mining, stochastic modeling, and forecasting, exposing a clear pipeline-fragmentation gap. **Objective.** We propose PATHCAST, a formally specified pipeline that integrates event log construction, process mining, absorbing Markov chain estimation, process-aware Monte Carlo simulation, and a machine-learning enhancement component into a single forecasting method for the SDLC. **Method.** The pipeline is defined as a composition of four sequential stages and a parallel enhancement component, each governed by formal input/output contracts. We prove pipeline correctness under stated preconditions and establish asymptotic consistency, end-to-end distributional convergence, and an information-theoretic advantage over process-blind throughput-based Monte Carlo. A closed-form numerical example on a six-state SDLC chain validates the analytical computations against simulation. **Contributions.** (i) A formally specified, contract-based PM-Markov-MC pipeline; (ii) proofs of correctness, statistical consistency and convergence; (iii) a novel family of PM- and Markov-derived features for delay prediction; (iv) a closed-form illustrative example with analytical-vs. Monte Carlo cross-validation. The companion empirical evaluation across 48 open-source repositories is reported separately.

Keywords: Process Mining, Stochastic Modeling, Absorbing Markov Chains, Monte Carlo Simulation, Software Process Forecasting, Software Development Lifecycle

1. Introduction

Software development lifecycle (SDLC) data is increasingly available in machine-readable form across version control, issue trackers, code-review systems and CI/CD platforms. Process mining (PM) and stochastic modeling each

*Corresponding author.

Email addresses: rodrigo1.almeida@pucpr.edu.br (Rodrigo Almeida de Oliveira), juliano.ribeiro@pucpr.edu.br (Juliano de Paulo Ribeiro), scalabrin@ppgia.pucpr.br (Edson Emilio Scalabrin)

[1]ORCID: 0009-0009-6310-4126

[2]ORCID: 0009-0004-3605-485X

[3]ORCID: 0000-0002-3918-1799

offer mature techniques for analyzing such data; yet their joint deployment for *predictive* use in the SDLC remains rare.

A recent systematic literature review across 5,783 deduplicated records and 404 confirmed primary studies [1] reports that only a single study jointly integrates process mining, stochastic modeling, and forecasting for the SDLC. The literature is descriptively rich but predictively fragmented: most works either discover process models without forecasting, forecast lead times without process awareness, or apply machine learning to features that ignore the underlying process structure.

This paper addresses the pipeline-fragmentation gap by proposing PATHCAST, a formally specified method that integrates event log construction, process discovery, absorbing Markov chain estimation, process-aware Monte Carlo simulation, and a machine-learning enhancement component. Our contributions are:

1. **C1.** A formal pipeline specification with contract-based stage composition (Section 3) and a correctness theorem under stated preconditions (Theorem 1).
2. **C2.** An empirically grounded stochastic model that couples a Markov transition kernel with empirical sojourn-time distributions (Section 4), preserving the distributional shape observed in real software processes.
3. **C3.** A novel family of features for delay prediction derived from PM and Markov analysis (Section 5), complementing traditional mining software repositories (MSR) features.
4. **C4.** Formal guarantees: asymptotic consistency, end-to-end distributional convergence, and an information-theoretic advantage over process-blind throughput-based Monte Carlo (Section 6).
5. **C5.** A closed-form numerical example (Section 7) that validates the analytical derivation against Monte Carlo simulation on a six-state SDLC chain.

This paper focuses on the *formal specification* of the method and its theoretical properties. The empirical evaluation of PATHCAST across 48 open-source repositories, including comparison with four baselines and statistical analysis of forecasting accuracy, is reported in a companion paper. The evaluation protocol is summarized in Section 9.

Section 2 reviews background and related work. Section 3 introduces the pipeline architecture and inter-stage contracts. Section 4 specifies the four sequential stages. Section 5 specifies the ML enhancement component. Section 6 establishes the formal properties. Section 7 presents the closed-form numerical example. Section 8 compares PATHCAST with related approaches. Section 9 outlines the planned empirical validation. Section 10 concludes.

2. Background and Related Work

2.1. Process Mining

Process mining [2] extracts process knowledge from event logs. The discipline distinguishes three families of techniques: discovery (e.g., the Inductive Miner [3]), conformance checking (alignments), and enhancement. The

Inductive Miner is adopted here for its formal soundness guarantee on the discovered process tree, which is required by Stage 2 of the pipeline (Section 4.2).

2.2. *Absorbing Markov Chains*

Absorbing Markov chains provide closed-form expressions for expected absorption times and absorption probabilities via the fundamental matrix $N = (I - Q)^{-1}$ [4]. Their application to business process analysis has been explored by Rogge-Solti and Weske [5] via stochastic Petri nets, and by [6] via direct estimation from event logs. PATHCAST adopts the latter approach but couples it with empirical sojourn-time distributions, avoiding the parametric assumptions of classical semi-Markov treatments.

2.3. *Stochastic Forecasting in Software Engineering*

Throughput-based Monte Carlo, popularized by Vacanti [7] and extended by [8, 9], samples future completion dates from the empirical throughput distribution of a team. The approach is operationally simple and widely adopted in agile practice, but treats the process as a black box: rework, state-specific delays, and path diversity are not captured. Recent work on probabilistic process forecasting in business processes includes [10, 11]; none of these methods is specialised to the SDLC nor integrates the heterogeneous data sources (Git, issue tracker, CI/CD, code review) characteristic of software repositories.

2.4. *Pipeline-Fragmentation Gap*

Table 1 summarizes the coverage of process mining, stochastic modeling, and forecasting across five representative categories of work in the SDLC literature. Only a single primary study in the SLR of [1] satisfies all three dimensions, motivating the integrated pipeline proposed in this paper.

Table 1: Coverage of PM, stochastic modeling, and forecasting across representative SDLC-related research families. "✓" indicates explicit treatment; "–" indicates absence.

Family	**PM**	**Stochastic**	**Forecasting**	**Representative**
SDLC process discovery	✓	–	–	[12]
Throughput Monte Carlo	–	✓	✓	[7]
Effort/lead-time ML	–	–	✓	[13]
Stochastic Petri nets (BPM)	✓	✓	–	[5]
PATHCAST (this paper)	✓	✓	✓	—

3. PATHCAST Architecture

3.1. Pipeline Definition

PATHCAST is defined as a composition of four sequential transformation stages and one parallel enhancement component:

Definition 1 (PATHCAST Pipeline)**.** *The PATHCAST pipeline is the function composition*

$$\mathcal{F} = \mathcal{S}_4 \circ \mathcal{S}_3 \circ \mathcal{S}_2 \circ \mathcal{S}_1 \tag{1}$$

where the stage signatures are

$$\mathcal{S}_1 : \mathcal{R} \to \mathcal{L} \qquad \textit{(Event Log Construction)} \tag{2}$$

$$\mathcal{S}_2 : \mathcal{L} \to (M, \Phi, S) \qquad \textit{(Process Mining)} \tag{3}$$

$$\mathcal{S}_3 : (\mathcal{L}, S) \to (P, N, B, \hat{F}, \pi) \qquad \textit{(Stochastic Modeling)} \tag{4}$$

$$\mathcal{S}_4 : (P, \hat{F}, s_0, M_{sim}) \to \hat{\mathcal{D}} \qquad \textit{(Monte Carlo Simulation)} \tag{5}$$

together with a parallel enhancement component

$$\mathcal{S}_{ML} : (\mathcal{L}, \Phi, P, N) \to (\hat{f}, \mathbf{w}) \quad \textit{(ML Enhancement)}. \tag{6}$$

Notation: $\mathcal{R}$ denotes the input repositories; $\mathcal{L}$ the structured event log; M the discovered process model; Φ the process metrics vector; $S = S_T \cup S_A$ the state space; P the Markov transition matrix; $N = (I - Q)^{-1}$ the fundamental matrix; $B = NR$ the absorption probability matrix; $\hat{F}$ the empirical sojourn-time distributions; π the stationary distribution; $\hat{\mathcal{D}}$ the empirical forecast distribution; $\hat{f}$ the trained ML predictor; $\mathbf{w}$ the feature-importance vector.

3.2. Pipeline Architecture

Figure 1 illustrates the pipeline with its data flows.

3.3. Design Principles

The architecture is governed by three principles. **(P1) Empirical grounding** — every model parameter is estimated from observed data; no *a priori* distribution is assumed unless empirically validated. **(P2) Stage composability** — each stage exposes a formal input/output contract (Table 2) so that stages may be independently tested, replaced or extended. **(P3) Distributional outputs** — the pipeline produces probability distributions, not point estimates; every forecast is the empirical distribution of M_{sim} simulated outcomes.

3.4. Inter-Stage Contracts

The contracts establish the preconditions of the correctness theorem proved in Section 6 (Theorem 1).

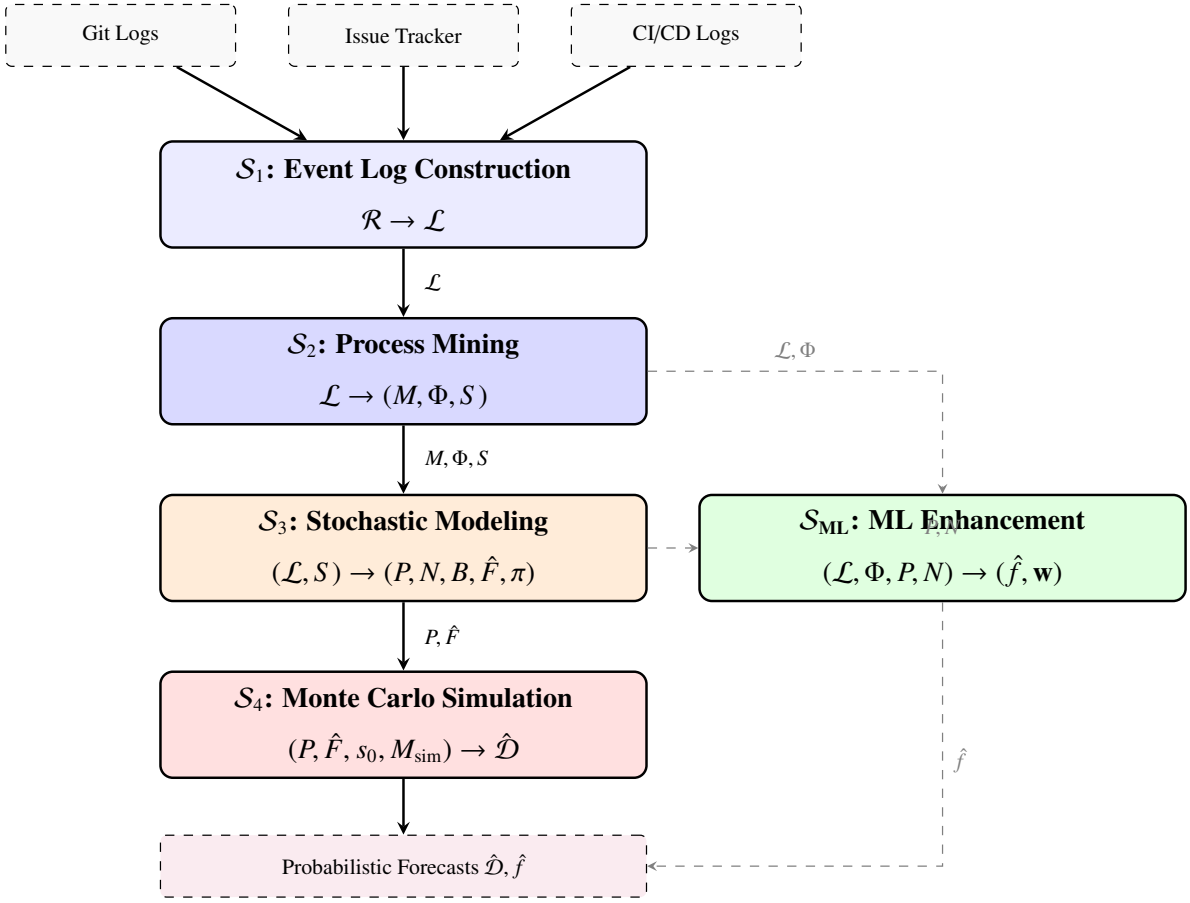

Figure 1: PATHCAST pipeline architecture. Solid arrows: primary sequential pipeline; dashed arrows: ML enhancement data flows.

Table 2: Inter-stage contracts. Each stage is specified by its input, output and key postconditions. Full specifications are given in Section 4.

Stage	Input	Output	Key Postconditions
$\mathcal{S}_1$	$\mathcal{R}$	$\mathcal{L}$	Valid case ID, activity, timestamp per event; timestamp-ordered within each trace.
$\mathcal{S}_2$	$\mathcal{L}$	(M, Φ, S)	M sound; $\phi_{\text{fit}}, \phi_{\text{prec}} \in [0,1]$; $\lvert S_T\rvert \geq 1$, $\lvert S_A\rvert \geq 1$.
$\mathcal{S}_3$	$(\mathcal{L}, S)$	$(P, N, B, \hat{F}, \pi)$	P row-stochastic; $(I - Q)$ invertible; $\sum_j B_{ij} = 1$.
$\mathcal{S}_4$	$(P, \hat{F}, s_0, M_{\text{sim}})$	$\hat{\mathcal{D}}$	$\lvert\hat{\mathcal{D}}\rvert = M_{\text{sim}}$; every trajectory absorbs or is flagged.
$\mathcal{S}_{\text{ML}}$	$(\mathcal{L}, \Phi, P, N)$	$(\hat{f}, \mathbf{w})$	Temporal hold-out enforced; AUC-ROC, F1, Brier reported.

4. Stage Specifications

4.1. Stage 1: Event Log Construction

Stage 1 transforms heterogeneous repository data into a structured event log. It is the composition

$$\mathcal{S}_1 = \textsc{Format} \circ \textsc{Map} \circ \textsc{Correlate} \circ \textsc{Extract}. \tag{7}$$

A raw event is a tuple $e^{\text{raw}} = (ts, src, type, payload)$ with timestamp, source, type and source-specific attribute map; the raw event pool is $E^{\text{raw}} = \bigcup_{i,j} \textsc{Extract}_{src(D_i^{(j)})}(D_i^{(j)})$ with source-specific extractors for Git, issue tracker, CI/CD and code review (full extraction/mapping tables in the online supplementary).

Case correlation.. The correlation function $\kappa : E^{\text{raw}} \to C \cup \{\perp\}$ is implemented as a cascade of three strategies in decreasing specificity: κ_1 explicit reference (issue keys in commit messages or PR descriptions),

$$\kappa_1(e) = \begin{cases} \text{issue\_key}(e.\text{payload}) & \text{if extractable} \\ \perp & \text{otherwise;} \end{cases} \tag{8}$$

κ_2 structural link (platform-provided PR→issue, build→commit, sub-task→parent); κ_3 heuristic (temporal proximity, author identity, file overlap). Correlation quality is monitored by

$$\text{Coverage} = |E^{\text{corr}}|/|E^{\text{raw}}| \tag{9}$$

$$\text{AmbiguityRate} = |\{e : |\kappa(e)| > 1\}|/|E^{\text{raw}}|. \tag{10}$$

Definition 2 (Structured Event)**.** *A structured event is* $e = (\kappa(e), \mu(e), \tau(e), \rho(e), \omega(e))$ *with case ID, activity, timestamp, resource, and attribute map.*

After bot filtering and per-triple $(case, activity, timestamp)$ deduplication, the output event log is

$$\mathcal{L} = \{\sigma_c = \langle e_1, \ldots, e_{|\sigma_c|}\rangle \;\big|\; c \in C,\ \tau(e_k) \le \tau(e_{k+1})\ \forall k\}\,. \tag{11}$$

4.2. Stage 2: Process Mining

Stage 2 produces a process model M, a metrics vector Φ and the state space $S = S_T \cup S_A$ used by Stage 3. The Inductive Miner [3] is applied with noise threshold $\theta_{\text{noise}} \in [0, 1]$:

$$M = \textsc{InductiveMiner}(\mathcal{L}, \theta_{\text{noise}}). \tag{12}$$

Property 1 (Soundness Guarantee)**.** *The Inductive Miner produces a sound process tree by construction [3].*

Alignment-based conformance produces $\phi_{\text{fit}}, \phi_{\text{prec}}, \phi_{\text{gen}} \in [0, 1]$ together with the per-case score

$$\phi_{\text{conf}}(c) = 1 - \frac{\text{cost}(\text{align}(\sigma_c, M))}{\text{cost}(\text{worst\_align}(\sigma_c, M))}, \tag{13}$$

which feeds feature 5 of the ML component (Section 5). Variant entropy $H_{\text{norm}}(V) = H(V)/\log_2|V| \in [0, 1]$ is computed as a scale-independent complexity proxy. The state space is $S = S_T \cup S_A$ with $S_A = \{\texttt{done}, \texttt{cancelled}\}$ and S_T the non-terminal observed activities; the per-state mean sojourn $\bar{d}_a$ feeds the expected lead time in Equation 17. Per-activity variance, median sojourn and per-transition waiting time are reported in the online supplementary.

4.3. Stage 3: Stochastic Modeling

Stage 3 is the formal core of PATHCAST: it estimates a transition matrix, an empirical sojourn-time distribution and the fundamental quantities of an absorbing Markov chain. The decomposition is

$$\mathcal{S}_3 = \textsc{Stationary} \circ \textsc{Sojourn} \circ \textsc{Absorb} \circ \textsc{Estimate} \circ \textsc{Count}. \tag{14}$$

Maximum-likelihood estimation with optional smoothing.. Let C_{ij} be the count of transitions from state i to state j observed in $\mathcal{L}$. The smoothed MLE estimator is

$$\hat{P}_{ij} = \frac{C_{ij} + \alpha}{\sum_{j'} C_{ij'} + \alpha|S|}, \qquad \alpha \ge 0, \tag{15}$$

with $\alpha = 0$ recovering the standard MLE. Symmetric smoothing ($\alpha > 0$) corresponds to a uniform Dirichlet prior on the row simplex and stabilizes estimates for low-count rows.

Property 2 (MLE Consistency)**.** *For $\alpha = 0$, $\hat{P} \xrightarrow{a.s.} P^*$ as $|\mathcal{L}| \to \infty$ [6].*

Absorbing-chain decomposition.. States are reordered so that transient precede absorbing:

$$\hat{P} = \begin{pmatrix} Q & R \\ \mathbf{0} & I \end{pmatrix}, \qquad N = (I - Q)^{-1}, \qquad B = NR, \tag{16}$$

yielding the expected number of visits matrix N and the absorption probability matrix B.

Property 3 (Invertibility)**.** *$(I - Q)$ is invertible iff every transient state can reach an absorbing state [4].*

Property 4 (Absorption Certainty)**.** $\sum_{a \in S_A} B_{ia} = 1$ *for every* $i \in S_T$.

Expected lead time in calendar time:

$$\mathbb{E}[\text{LeadTime} \mid s_0 = i] = \sum_{j \in S_T} N_{ij} \cdot \bar{d}_j. \tag{17}$$

Empirical sojourn-time distributions.. For each $s \in S_T$, the observed sojourn set is $\mathcal{T}_s = \{\tau(e_{k+1}) - \tau(e_k) \mid \mu(e_k) = s\}$ with empirical CDF

$$\hat{F}_s(t) = \tfrac{1}{|\mathcal{T}_s|} \sum_{\tau \in \mathcal{T}_s} \mathbf{1}[\tau \le t]. \tag{18}$$

Remark 1 (Non-Parametric Design Choice)**.** *Empirical resampling preserves the multi-modal, heavy-tailed and zero-inflated patterns commonly observed in software process sojourns, avoiding the misspecification incurred by lognormal, Weibull or gamma fits [7].*

Confidence assessment.. For row i with $n_i = \sum_j C_{ij}$ observations,

$$\text{SE}(\hat{P}_{ij}) = \sqrt{\hat{P}_{ij}(1 - \hat{P}_{ij})/n_i}. \tag{19}$$

Rows with $n_i < n_{\min}$ (default 30) are flagged; the row-level confidence grade is

$$\text{Grade}(P) = |\{i \in S_T : n_i \ge n_{\min}\}|/|S_T|, \tag{20}$$

and $\text{Grade}(P) < 0.8$ is reported as a validity caveat. The quasi-stationary distribution and rework-intensity decomposition are reported in the online supplementary; full sensitivity to α is analysed there.

4.4. Stage 4: Process-Aware Monte Carlo Simulation

Stage 4 generates the empirical forecast distribution by simulating trajectories of the absorbing chain with sojourn times sampled from $\hat{F}_s$:

$$\hat{\mathcal{D}} = \{(T^{(m)}, o^{(m)}, \psi^{(m)})\}_{m=1}^{M_{\text{sim}}}. \tag{21}$$

The safety bound $K_{\max} = 1000$ prevents infinite loops in degenerate cases; non-absorbed trajectories are excluded from forecast aggregation. PATHCAST uses Mersenne Twister (MT19937) as the default PRNG and records seeds per run for exact replication.

Forecast extraction.. From $\hat{\mathcal{D}}$ the pipeline extracts the percentile forecasts

$$\hat{Q}_\alpha = \inf\{t : \hat{F}_T(t) \ge \alpha\} \quad (\alpha \in \{0.5, 0.85, 0.95\}), \tag{22}$$

the completion probability

$$\hat{P}(\text{Done}) = |\{m : o^{(m)} = \texttt{done}\}|/M_{\text{sim}}, \tag{23}$$

and the tail risk

$$\hat{P}(\text{late}) = |\{m : T^{(m)} > \tau_{\text{deadline}}\}|/M_{\text{sim}}. \tag{24}$$

Algorithm 1 Process-Aware Monte Carlo Simulation

Require: P, $\hat{F}$, π_0 (or fixed s_0), M_{sim}, max steps $K_{\max}$

Ensure: $\hat{\mathcal{D}} = \{(T^{(m)}, o^{(m)}, \psi^{(m)})\}_{m=1}^{M_{\text{sim}}}$

$\hat{\mathcal{D}} \leftarrow \emptyset$

for $m = 1$ to M_{sim} **do**

  $s \leftarrow s_0$ if fixed, else $s \leftarrow$ sample from π_0

  $T \leftarrow 0$; $\psi \leftarrow [s]$; $k \leftarrow 0$

  while $s \notin S_A$ **and** $k < K_{\max}$ **do**

    $\Delta t \leftarrow$ sample from $\hat{F}_s$

    $T \leftarrow T + \Delta t$

    $s' \leftarrow \textsc{Categorical}(P[s, :])$

    $s \leftarrow s'$; append s to ψ; $k \leftarrow k + 1$

  end while

  if $s \notin S_A$ **then**

    mark trajectory m as `non-absorbed`

  end if

  $\hat{\mathcal{D}} \leftarrow \hat{\mathcal{D}} \cup \{(T, s, \psi)\}$

end for

return $\hat{\mathcal{D}}$

Convergence..

Definition 3 (Convergence Criterion)**.** *The simulation budget M_{sim} satisfies the convergence criterion at tolerance ϵ when*

$$\frac{z_{0.025} \cdot \hat{\sigma}_T / \sqrt{M_{sim}}}{\hat{Q}_{0.5}} < \epsilon. \tag{25}$$

Property 5 (Convergence Rate)**.** *The MC estimation error decreases as $O(1/\sqrt{M_{sim}})$ [14].*

Adaptive doubling from $M_{\text{sim}} = 1{,}000$ to 100,000 stops when Equation 25 holds with $\epsilon = 0.02$.

5. Machine-Learning Enhancement Component

The ML component $\mathcal{S}_{\text{ML}}$ runs in parallel to the sequential pipeline and produces a case-level predictor $\hat{f}$. Its contract is summarized in Table 2; its novelty is the *family of features* it consumes, half of which are PM- and Markov-derived and are not available to traditional MSR baselines.

5.1. Feature Engineering

Table 3: Feature definitions. Features 1–8 are derived from the PM and Markov stages; features 9–15 are traditional MSR features used as baseline.

#	Feature	Source	Definition
Process-derived (novel)			
1	Current state	$\mathcal{L}$	$\mu(e_{\text{last}})$
2	Sojourn in current state	$\mathcal{L}$	$t_{\text{obs}} - \tau(e_{\text{last}})$
3	Transition count	$\mathcal{L}$	$\lvert\sigma_c^{\text{partial}}\rvert$
4	Rework count	$\mathcal{L}$	regression count
5	Conformance score	$M, \mathcal{L}$	$\phi_{\text{conf}}(c)$
6	Markov expected remaining	N	$t_{\mu(e_{\text{last}})}$
7	Completion probability	B	$B_{\mu(e_{\text{last}}),\texttt{done}}$
8	Sojourn ratio	$\hat{F}$	sojourn/$\bar{d}_{\mu(e_{\text{last}})}$
Traditional MSR (baseline)			
9–15	Files / lines / commits / authors / priority / type / etc.	Git, Jira	—

Three feature sets are evaluated in the companion empirical paper: **PM-only** (1–8), **MSR-only** (9–15), and **Combined** (1–15).

5.2. Label Construction and Training Protocol

Definition 4 (Outcome Label)**.** $y_c = 1$ *if* $T_c > \tau_{threshold}$ (`delayed`)*;* $y_c = 0$ *otherwise* (`on_time`)*.*

Three threshold strategies are supported: *percentile-based* ($\tau_{\text{threshold}} = \hat{Q}_{0.75}$), *SLA-based*, and *relative* ($2 \cdot \tilde{d}_{\text{type}}$).

The training protocol enforces a strict temporal hold-out: $\mathcal{L}_{\text{train}}$ contains all cases completed before t_{cut}, $\mathcal{L}_{\text{test}}$ contains only cases that *start* after t_{cut}. Candidate models $\{\text{RF}, \text{XGBoost}, \text{LogReg}\}$ are compared by temporal cross-validation with expanding windows [15, 16]; the best model $\hat{f}$ is evaluated on $\mathcal{L}_{\text{test}}$ via AUC-ROC, F1, Brier score, and the feature-importance vector $\mathbf{w}$ is reported.

5.3. Integration with Monte Carlo Forecasts

Definition 5 (Deviation Score)**.**

$$\delta_c = \hat{f}(\mathbf{x}_c) - \hat{P}_{MC}(\textit{delayed} \mid s_0 = s). \tag{26}$$

Monte Carlo (§4.4) produces population-level distributional forecasts; the ML component produces case-level point forecasts; δ_c identifies cases departing from the population baseline.

6. Formal Properties

6.1. Pipeline Correctness

Theorem 1 (Pipeline Correctness)**.** *If each stage satisfies its contract (Table 2), then the composed pipeline* $\mathcal{F} = \mathcal{S}_4 \circ \mathcal{S}_3 \circ \mathcal{S}_2 \circ \mathcal{S}_1$ *produces a valid empirical forecast distribution* $\hat{\mathcal{D}}$ *for any input* $\mathcal{R}$ *satisfying: (i) at least one repository contains events from* ≥ 2 *data sources; (ii)* $|\mathcal{L}| \geq n_{\min}^{cases}$ *(default 50); (iii)* $|S_T| \geq 1$ *and* $|S_A| \geq 1$.

Proof. By structural induction on stages: each postcondition implies the precondition of the next stage. (S1→S2) event-log validity; (S2→S3) sound model and non-empty state space; (S3→S4) row-stochastic P and non-empty empirical sojourn distributions; S4 terminates either by absorption certainty (Property 4) or by the $K_{\max}$ safety bound. □ □

6.2. Statistical Consistency

Theorem 2 (Asymptotic Consistency)**.** *As* $|\mathcal{L}| \to \infty$ *and* $M_{sim} \to \infty$*:*

(i) $\hat{P} \xrightarrow{a.s.} P^*$;

(ii) $\hat{F}_T \xrightarrow{d} F_T^*$;

(iii) $\hat{Q}_\alpha \to Q_\alpha^*$ *at continuity points.*

Proof sketch. (i) Property 2. (ii) Continuous mapping theorem applied to (i) and the Glivenko–Cantelli convergence $\hat{F}_s \to F_s^*$. (iii) Quantile continuity. □ □

6.3. End-to-End Distributional Convergence

Definition 6 (Forecast Calibration)**.** *For a finite set of nominal levels $\mathcal{A} \subset (0, 1)$,*

$$CalErr(\alpha) = |\hat{P}(T \le \hat{Q}_\alpha) - \alpha|, \qquad ACE = \frac{1}{|\mathcal{A}|} \sum_{\alpha \in \mathcal{A}} CalErr(\alpha). \tag{27}$$

Theorem 3 (End-to-End Convergence)**.** *Under (a) ergodicity of the underlying chain, (b) uniform sojourn consistency* $\sup_t |\hat{F}_i^{(n)}(t) - F_i^*(t)| \xrightarrow{a.s.} 0$ *for each i, and (c)* $K(n) \to \infty$*:*

$$\sup_t |\hat{F}_{n,K}(t) - F^*(t)| \xrightarrow{a.s.} 0, \qquad ACE_{n,K} \to 0. \tag{28}$$

Proof sketch. SLLN for ergodic Markov chains gives entry-wise $\hat{P}^{(n)} \to P$ [17]; Glivenko–Cantelli per state plus union bound across the finite S; continuous mapping on simulated trajectories plus a second Glivenko–Cantelli application across K replications gives the uniform convergence. ACE convergence follows. □

6.4. Information-Theoretic Advantage of Process Awareness

Theorem 4 (Information Gain from Process Structure)**.**

$$I(T; \psi) = H(T) - H(T \mid \psi) \ge 0, \tag{29}$$

with equality iff T is independent of the trajectory ψ.

Practical interpretation.. In real software processes $I(T; \psi) > 0$ because rework-laden paths have systematically longer lead times: the trajectory ψ carries information about state-specific delays and revisits that the marginal T does not. Process-aware Monte Carlo (Stage 4) samples the joint (T, ψ) and captures this variance decomposition; throughput-based Monte Carlo collapses ψ into a daily-throughput scalar and forfeits the information gain.

6.5. Complexity and Assumptions

The per-stage time complexity is summarized in Table 4; the total cost is dominated by Stage 4:

$$C_{\text{total}} = O(|\mathcal{L}|) + O(M_{\text{sim}} \cdot \bar{K}), \tag{30}$$

where $\bar{K}$ is the expected trajectory length, bounded by $K_{\max}$. Monte Carlo is embarrassingly parallel and admits GPU offload [18].

The theoretical results rest on five explicit assumptions, listed in Table 5 together with the consequence of violation and the mitigation adopted by PATHCAST.

7. Illustrative Numerical Example

This section instantiates PATHCAST on a controlled six-state SDLC chain to demonstrate (i) the closed-form analytical derivation of N, B and the expected lead time and (ii) agreement with the Monte Carlo simulation of Stage 4. The parameters are synthetic and chosen to mirror typical SDLC patterns; no real-repository data is involved.

Table 4: Per-stage time complexity.

Stage	**Time complexity**	**Dominant factor**
$\mathcal{S}_1$: Event Log	$O(\lvert E^{\text{raw}}\rvert \cdot \lvert\mathcal{K}\rvert)$	raw events × strategies
$\mathcal{S}_2$: Discovery + Conf.	$O(\lvert\mathcal{L}\rvert \cdot \lvert\mathcal{A}\rvert^2 + \lvert C\rvert \cdot \lvert M\rvert)$	alignment cost
$\mathcal{S}_3$: Markov	$O(\lvert\mathcal{L}\rvert + \lvert S_T\rvert^3)$	counting + matrix inversion
$\mathcal{S}_4$: Monte Carlo	$O(M_{\text{sim}} \cdot \bar{K})$	simulations × path length
$\mathcal{S}_{\text{ML}}$: Training	$O(\lvert C\rvert \cdot p \cdot T)$	cases × features × trees

Table 5: Assumptions, violation consequences and mitigations.

Assumption	**Violation consequence**	**Mitigation**
A1 First-order Markov	Underestimates rework-conditioned delays	State augmentation; ML features
A2 Stationarity	Stale forecasts after process change	Sliding window; drift detection
A3 Case independence	Ignores queueing/resource contention	Acknowledged limitation
A4 Sojourn-history independence	Underestimates reworked-item delays	State augmentation; ML features
A5 Complete case observation	Bias if long cases excluded	Include right-censored cases

7.1. Setup

The state space is $S = S_T \cup S_A$ with $S_T = \{\texttt{backlog}, \texttt{in\_progress}, \texttt{review}, \texttt{testing}\}$ and $S_A = \{\texttt{done}, \texttt{cancelled}\}$. The transition matrix P encodes the nominal forward flow plus rework edges from `review` and `testing` back to `in_progress` and a small cancellation probability from `backlog` and `in_progress`:

$$P = \begin{pmatrix} 0.0 & 0.9 & 0.0 & 0.0 & 0.0 & 0.1 \\ 0.0 & 0.0 & 0.8 & 0.0 & 0.0 & 0.2 \\ 0.0 & 0.3 & 0.0 & 0.6 & 0.1 & 0.0 \\ 0.0 & 0.2 & 0.0 & 0.0 & 0.8 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}.$$

Per-state mean sojourn (in days): $\bar{d}_{\texttt{backlog}} = 3$, $\bar{d}_{\texttt{in\_progress}} = 4$, $\bar{d}_{\texttt{review}} = 2$, $\bar{d}_{\texttt{testing}} = 3$.

7.2. Closed-Form Computation

Ordering states as (`backlog`, `in_progress`, `review`, `testing`, `done`, `cancelled`) puts P in canonical form (Equation 16) with

$$Q = \begin{pmatrix} 0 & 0.9 & 0 & 0 \\ 0 & 0 & 0.8 & 0 \\ 0 & 0.3 & 0 & 0.6 \\ 0 & 0.2 & 0 & 0 \end{pmatrix}, \qquad R = \begin{pmatrix} 0 & 0.1 \\ 0 & 0.2 \\ 0.1 & 0 \\ 0.8 & 0 \end{pmatrix}.$$

Inverting $I - Q$ in double-precision arithmetic yields the fundamental matrix

$$N = (I - Q)^{-1} \approx \begin{pmatrix} 1.000 & 1.355 & 1.084 & 0.651 \\ 0 & 1.506 & 1.205 & 0.723 \\ 0 & 0.633 & 1.506 & 0.904 \\ 0 & 0.301 & 0.241 & 1.145 \end{pmatrix},$$

and the absorption matrix

$$B = NR \approx \begin{pmatrix} 0.629 & 0.371 \\ 0.699 & 0.301 \\ 0.873 & 0.127 \\ 0.940 & 0.060 \end{pmatrix},$$

whose rows sum to one (Property 4).

Expected lead time from $s_0 = \texttt{backlog}$ (Equation 17):

$$\begin{aligned} \mathbb{E}[\text{LeadTime} \mid s_0 = \texttt{backlog}] &= 1.000 \cdot 3 + 1.355 \cdot 4 + 1.084 \cdot 2 + 0.651 \cdot 3 \\ &\approx 12.54 \text{ days}. \end{aligned} \tag{31}$$

Completion probability from `backlog`:

$$B_{\texttt{backlog},\texttt{done}} \approx 0.629, \qquad B_{\texttt{backlog},\texttt{cancelled}} \approx 0.371. \tag{32}$$

Expected number of transient visits before absorption is $t_{\texttt{backlog}} = (N\mathbf{1})_{\texttt{backlog}} \approx 4.09$.

7.3. Monte Carlo Cross-Validation

We instantiate Stage 4 (Algorithm 1) with P above, exponential sojourns matched to $\bar{d}_s$, $s_0 = \texttt{backlog}$, and $M_{\text{sim}} = 10{,}000$. Table 6 compares the closed-form quantities against their Monte Carlo estimates; the relative error is below 1% on every quantity, as predicted by Property 5.

Table 6: Analytical vs. Monte Carlo estimates on the six-state SDLC chain (s_0 = `backlog`, M_{sim} = 10,000).

Quantity	**Analytical**	**Monte Carlo**	**Rel. error**
$\mathbb{E}$[LeadTime | `backlog`]	12.54 d	12.51 d	0.24%
$\mathbb{E}$[Steps | `backlog`]	4.090	4.097	0.17%
$\hat{P}$(`done` | `backlog`)	0.629	0.631	0.32%

7.4. Discussion

The example illustrates three properties of the pipeline. **(D1)** The fundamental matrix N exposes the contribution of each transient state to the expected lead time; the row $N_{\texttt{backlog},\cdot}$ shows that `in_progress` (visited 1.355 times) and `review` (visited 1.084 times) account for 7.59 of the 12.54 expected days through the rework loop `review` → `in_progress`. **(D2)** The non-trivial cancellation mass $B_{\texttt{backlog},\texttt{cancelled}} \approx 0.37$ is a direct output of the absorbing-chain decomposition, unavailable to throughput-based Monte Carlo, which produces a single completion distribution. **(D3)** Analytical and Monte Carlo answers agree within $\sim$ 0.3%, a controlled cross-validation that the simulation implements the same stochastic model as the closed-form derivation.

This example is not an empirical evaluation. The empirical evaluation across 48 open-source repositories is the subject of the companion paper (Section 9).

8. Theoretical Comparison with Related Approaches

We compare PATHCAST with three families of related approaches along five theoretical dimensions: process model, sampling unit, rework modeling, state-specific delays, and distributional outputs.

Table 7: Theoretical comparison of forecasting approaches along five dimensions. "–" indicates the approach does not capture the dimension.

Dimension	**Throughput MC** [7]	**Markov analyt.** [4]	**NM-SPN** [5]	**PATHCAST**
Process model	–	exogenous	Petri net	PM-discovered
Sampling unit	daily throughput	none (analytical)	timed firing	transition + sojourn
Rework	–	via N	via Petri loops	via P_{ab} + $\hat{F}$
State-specific delay	–	via $\bar{d}_s$	general dist.	empirical $\hat{F}_s$
Distributional output	completion only	–	lead-time only	LT + outcome + path + WIP

Throughput Monte Carlo.. Throughput MC samples future cumulative completion from the empirical throughput distribution of a team. It is operationally simple but process-blind: rework, state-specific delays, and path information are not represented. By Theorem 4, throughput MC cannot exploit the information gain $I(T;\psi)$ encoded in the trajectory.

Analytical absorbing Markov chains.. The closed-form expectations $N\mathbf{1}$ and NR are deterministic; they yield expected lead time but not a forecast distribution. PATHCAST uses these expressions for analytical cross-validation (Section 7) but produces distributional forecasts via Monte Carlo, exposing percentiles and tail risk that the analytical baseline cannot.

Non-Markovian stochastic Petri nets (NM-SPN).. NM-SPN [5] parameterize timed firings with general distributions over an exogenously specified Petri net. PATHCAST differs in two respects: (i) the process model is *discovered* from the event log rather than supplied externally, removing a strong modelling burden; (ii) sojourn distributions are empirical rather than parametric, avoiding misspecification of the heavy-tailed, zero-inflated distributions typical of software processes.

Process-prediction approaches.. The predictive process monitoring literature [10, 11] addresses remaining time prediction in business processes, typically via supervised learning. PATHCAST occupies a complementary niche: it integrates a stochastic process model (not a black-box predictor) and is specialised to the heterogeneous data sources of the SDLC. The ML component (Section 5) bridges to this literature by adding case-level point forecasts on top of the population-level distribution.

9. Evaluation Plan

The empirical validation of PATHCAST is reported in a companion paper (under review). For completeness, we summarize the design here.

The protocol follows Design Science Research [19, 20] and is evaluated on a dataset of 48 open-source repositories selected by stratified sampling across primary language, project scale, governance model, and workflow maturity, totalling approximately 227,000 commits. The experimental design comprises two stages: **E1 (pipeline validation)**, which assesses event-log and process-model quality (correlation coverage, activity coverage, fitness, precision, generalization); and **E2 (forecasting accuracy)**, which compares PATHCAST against four baselines — historical mean, throughput-based Monte Carlo [7], analytical Markov, and historical empirical CDF — using the continuous ranked probability score (CRPS) as primary metric, mean absolute error (MAE) and median absolute error for point accuracy, the aggregate calibration error (ACE) for distributional fidelity, and 85% coverage for the operational reporting interval. The ML component is evaluated with AUC-ROC, F1, and Brier score across three feature sets (PM-only, MSR-only, Combined). Statistical analysis uses Wilcoxon signed-rank tests with Bonferroni correction ($\alpha_{\text{adj}} = 0.0125$) across repositories and Cliff's δ for effect size [21]. Temporal hold-out (70/30 split) and leave-one-project-out within domain groups control for information leakage and cross-repository generalization.

This paper does not report empirical results. The reader is referred to the companion paper for the full evaluation, including calibration plots, CRPS box plots, and feature-importance rankings.

10. Conclusion

We introduced PATHCAST, a formally specified pipeline for process-aware stochastic forecasting of the software development lifecycle. The pipeline integrates event log construction, process mining, absorbing Markov chain estimation, process-aware Monte Carlo simulation, and a machine-learning enhancement component, under contract-based stage composition. We proved pipeline correctness under stated preconditions (Theorem 1), asymptotic statistical consistency (Theorem 2), end-to-end distributional convergence (Theorem 3), and an information-theoretic advantage of process awareness over throughput-based Monte Carlo (Theorem 4). A closed-form numerical example on a six-state SDLC chain cross-validates the analytical and Monte Carlo computations.

The theoretical results rest on five explicit assumptions: (A1) first-order Markov dynamics, (A2) stationarity, (A3) case independence, (A4) sojourn–history independence, and (A5) complete case observation. Each assumption is reported with its mitigation (state augmentation, sliding-window estimation, right-censored inclusion). The relaxation of A1 via second-order or history-augmented chains is a natural extension.

The companion empirical paper reports the full evaluation across 48 open-source repositories with comparison to four baselines. Two further directions are anticipated: (i) integration of resource and queueing effects (relaxing A3) for team-level scheduling; (ii) online operation with drift detection on P and $\hat{F}$ for continuous forecasting.

CRediT authorship contribution statement

Rodrigo Almeida de Oliveira: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. **Juliano de Paulo Ribeiro:** Methodology, Validation, Writing – review & editing. **Edson Emilio Scalabrin:** Conceptualization, Methodology, Supervision, Writing – review & editing, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

Funding

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior — Brasil (CAPES) — Finance Code 001, through a doctoral scholarship granted to the corresponding author within the Graduate Program in Informatics at the Pontifícia Universidade Católica do Paraná (PPGIa/PUC-PR).

Data availability

A replication package containing the numerical example, parameter files, and reference implementation of the pipeline stages is deposited at Zenodo (DOI 10.5281/zenodo.XXXXXXX). The empirical validation dataset and results are reported in a companion paper.

References

[1] R. Almeida de Oliveira, J. de Paulo Ribeiro, E. E. Scalabrin, Process mining and stochastic modeling for software process forecasting: A systematic literature review with an llm-assisted protocol, Information and Software TechnologyUnder review (2026).

[2] W. M. P. van der Aalst, Process Mining: Data Science in Action, 2nd Edition, Springer, 2016.

[3] S. J. J. Leemans, D. Fahland, W. M. P. van der Aalst, Discovering block-structured process models from event logs — a constructive approach, in: Application and Theory of Petri Nets and Concurrency, Springer, 2013, pp. 311–329.

[4] J. G. Kemeny, J. L. Snell, Finite Markov Chains, Springer, 1976.

[5] A. Rogge-Solti, M. Weske, Prediction of business process durations using non-Markovian stochastic Petri nets, Information Systems 54 (2015) 1–14.

[6] G. A. Spedicato, Discrete time markov chains with r, The R Journal 9 (2) (2017) 84–104.

[7] D. S. Vacanti, Actionable Agile Metrics for Predictability: An Introduction, Leanpub, 2015.

[8] S. L. Savage, The Flaw of Averages: Why We Underestimate Risk in the Face of Uncertainty, Wiley, 2009.

[9] T. Magennis, Forecasting and simulating software development projects: Effective modeling of Kanban & Scrum projects using Monte carlo simulation (2015).

[10] M. Polato, A. Sperduti, A. Burattin, M. de Leoni, Time and activity sequence prediction of business process instances, Computing 100 (9) (2018) 1005–1031.

[11] I. Verenich, M. Dumas, M. La Rosa, F. M. Maggi, I. Teinemaa, Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring, ACM Transactions on Intelligent Systems and Technology 10 (4) (2019).

[12] V. Rubin, C. W. Günther, W. M. P. van der Aalst, E. Kindler, B. F. van Dongen, W. Schäfer, Process mining framework for software processes, in: International Conference on Software Process, 2007.

[13] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, T. Menzies, A deep learning model for estimating story points, IEEE Transactions on Software Engineering 45 (7) (2018) 637–656.

[14] C. P. Robert, G. Casella, Monte Carlo Statistical Methods, 2nd Edition, Springer, 2004.

[15] L. J. Tashman, Out-of-sample tests of forecasting accuracy: An analysis and review, International Journal of Forecasting 16 (4) (2000) 437–450.

[16] R. J. Hyndman, G. Athanasopoulos, Forecasting: Principles and Practice, 3rd Edition, OTexts, 2021.

[17] P. Billingsley, Statistical methods in Markov chains, The Annals of Mathematical Statistics 32 (1) (1961) 12–40.

[18] P. L'Ecuyer, D. Munger, B. Oreshkin, R. Simard, Random numbers for parallel computers: Requirements and methods, with emphasis on GPUs, Mathematics and Computers in Simulation 135 (2017) 3–17.

[19] A. R. Hevner, S. T. March, J. Park, S. Ram, Design science in information systems research, MIS Quarterly 28 (1) (2004) 75–105.

[20] K. Peffers, T. Tuunanen, M. A. Rothenberger, S. Chatterjee, A design science research methodology for information systems research, Journal of Management Information Systems 24 (3) (2007) 45–77.

[21] V. B. Kampenes, T. Dybå, J. E. Hannay, D. I. K. Sjøberg, A systematic review of effect size in software engineering experiments, Information and Software Technology 49 (11–12) (2007) 1073–1086.